

iFairy: the First 2-bit Complex LLM with All Parameters in $\{\pm 1, \pm i\}$

Feiyu Wang, Guoan Wang, Yihao Zhang, Shengfan Wang, Weitao Li, Bokai Huang,
Shimao Chen, Zihan Jiang, Rui Xu, Tong Yang*

Peking University

Model-700M: <https://huggingface.co/PKU-DS-LAB/Fairy-plus-minus-i-700M>

Model-1.3B: <https://huggingface.co/PKU-DS-LAB/Fairy-plus-minus-i-1.3B>

Code: <https://github.com/PKULab1806/Fairy-plus-minus-i>

Keywords: *iFairy*, *Fairy* $\pm i$, *Fairy*-imaginary

Abstract

Quantization-Aware Training (QAT) integrates quantization into the training loop, enabling LLMs to learn robust low-bit representations, and is widely recognized as one of the most promising research directions. All current QAT research focuses on minimizing quantization error on full-precision models, where the full-precision accuracy acts as an upper bound (accuracy ceiling). No existing method has even attempted to surpass this ceiling. To break this ceiling, we propose a new paradigm: raising the ceiling (full-precision model), and then still quantizing it efficiently into 2 bits. We propose *iFairy*, the first 2-bit quantization framework for complex-valued LLMs. Specifically, our method leverages the representational advantages of the complex domain to boost full-precision accuracy. We map weights to the fourth roots of unity $\{\pm 1, \pm i\}$, forming a perfectly symmetric and information-theoretically optimal 2-bit representation. Importantly, each quantized weight has either a zero real or imaginary part, enabling multiplication-free inference using only additions and element swaps. Experimental results show that *iFairy* outperforms the ceiling of existing 2-bit quantization approaches in terms of both PPL and downstream tasks, while maintaining strict storage and compute efficiency. This work opens a new direction for building highly accurate and practical LLMs under extremely low-bit constraints.

1 Introduction

The advent of Large Language Models (LLMs) has transformed artificial intelligence, achieving remarkable performance across a wide range of natural language tasks (Achiam et al. 2023; Touvron et al. 2023; Dubey et al. 2024). However, the deployment of these models in real-world applications faces two critical bottlenecks: **spatial** and **temporal**. The spatial bottleneck arises from the massive number of parameters, which is often in the billions or trillions, leading to prohibitive storage requirements and large memory footprints. The temporal bottleneck, on the other hand, stems from the heavy reliance on large-scale matrix multiplications during inference, which not only slows down computation but also significantly increases power consumption. Overcoming these two bottlenecks by maintaining model accuracy under extreme compression and reducing or eliminating costly multiplications would greatly

enhance the efficiency of LLMs, enabling transformative applications in domains such as physics, chemistry, biology, astronomy, and geoscience.

Model compression has thus become a critical research area, with *quantization* emerging as one of the most promising techniques to alleviate these bottlenecks (Miao et al. 2023; Wan et al. 2023). Quantization methods are broadly categorized into Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). While PTQ (Frantar et al. 2022; Lin et al. 2023) offers simplicity, its performance often degrades sharply in extremely low-bit scenarios due to the model’s lack of adaptation to quantized representations. In contrast, QAT integrates quantization into the training loop, allowing models to learn robust low-bit representations and maintain performance under aggressive compression. This advantage has motivated recent research into QAT-based strategies tailored for LLMs.

The pursuit of extremely low-bit quantization, particularly 2-bit quantization, has become a focal point in efforts to compress Large LLMs for efficient deployment. Existing approaches, such as BitNet (Wang et al. 2023) and its successors (Ma et al. 2024), have demonstrated that it is possible to retain reasonable accuracy using ternary quantization schemes with just 1.58 bits per weight. However, the accuracy of any quantized model is fundamentally limited by the following equation:

$$\mathbf{Accuracy}_{\text{quant}} = \mathbf{Accuracy}_{\text{full-precision}} - \mathbf{Error}_{\text{quant}}$$

All current quantization research focuses on minimizing quantization error on full-precision models (e.g., LLaMA), but the quantization error can never be zero. Therefore, full-precision accuracy becomes the **ceiling** for quantized accuracy. To date, no existing method has even attempted to surpass this ceiling.

In this paper, we propose a fundamentally different perspective. Instead of solely focusing on reducing quantization error, we make the first attempt to raise the ceiling (the accuracy of the full-precision model), while still ensuring that the resulting model can be efficiently quantized to a 2-bit format. Our key insight is that if the full-precision model becomes more expressive and accurate, the final 2-bit quantized model can achieve higher accuracy as well. Building on this insight, we propose, for the first time, incorporating complex-valued neural architectures into LLMs. The complex number provides a richer representational space with

*Corresponding author: yangtong@pku.edu.cn

additional phase information, thereby enhancing the expressiveness of linear transformations without increasing the parameter count. By systematically extending the Transformer architecture into the complex domain, we construct a full-precision complex-valued LLM with superior modeling capacity.

Building upon this complex-valued foundation, we further design a novel 2-bit quantization scheme tailored for complex weights. Specifically, we quantize each complex parameter to one of the **fourth roots of unity** $\{\pm 1, \pm i\}$ in the complex plane. This approach, unlike real-valued quantization, exploits the full 2-bit representational capacity *without sacrificing symmetry or sparsity*, thereby eliminating the trade-offs that limit real-valued schemes. The resulting model, which we name *iFairy* (also named *Fairy $\pm i$* , *Fairy-imaginary*), is perfectly storage-efficient and phase-aware by design. We propose a quantization function *PhaseQuant* that learns to project full-precision complex weights onto the target set $\{\pm 1, \pm i\}$ while preserving both magnitude and phase information. We implement this within our complex Transformer framework and evaluate its performance under the same storage and compute constraints as BitNet b1.58. Experiments show that *iFairy* significantly improves perplexity and downstream task accuracy, outperforming existing 2-bit baselines and approaching the performance of full-precision FP16 models.

Our contributions can be summarized as follows:

- We propose a new perspective on low-bit quantization: improving the accuracy of quantized models by raising the ceiling (the full precision model).
- We design a complex-valued LLM architecture that leverages the representational benefits of the complex domain without increasing parameter storage.
- We design a 2-bit quantization scheme that maps complex weights to the 4th roots of unity $\{\pm 1, \pm i\}$, fully utilizing bit capacity while preserving key properties like symmetry and sparsity.
- Experimental results show that our quantized model outperforms the ceiling of existing 2-bit quantization approaches in terms of both PPL and downstream understanding tasks.

2 Related Work

2.1 Quantization Techniques

The effort to mitigate the computational burden of large language models has led to significant research in model quantization. Quantization aims to reduce memory footprint and computation cost by representing weights, activations, and occasionally gradients using low-precision numerical formats. Existing quantization methods are typically categorized into post-training quantization (PTQ) and quantization-aware training (QAT). Post-training quantization (PTQ) applies quantization to a pretrained full-precision model without additional training. Notable methods include GPTQ (Frantar et al. 2022), a one-shot quantization algorithm that leverages approximate second-order information; AWQ (Lin et al. 2023), introducing channel-wise weight

quantization along with activation weighting to improve output calibration; and SmoothQuant (Xiao et al. 2023), which jointly scales weights and activations to enable robust 8-bit quantization. These PTQ techniques have shown remarkable performance with minimal degradation. Quantization-aware training (QAT), in contrast, integrates quantization directly into the training process. During both forward and backward passes, quantized values are used to allow the model to adapt to quantization-induced constraints. QAT generally yields better accuracy than PTQ, especially for sub-4-bit precision and end-to-end quantized models. While QAT introduces additional training overhead, it enables more accurate models. Typical QAT works include (Liu et al. 2023; Chen et al. 2024; Bondarenko, Del Chiaro, and Nagel 2024). In this work, we propose a novel QAT-based quantization framework specifically designed for extremely low-bit complex-valued language models.

2.2 Extremely Low-Bit LLMs

To reduce the storage and computational costs of large-scale models, many studies have extended binary neural network techniques to large language models. Early works on binary neural networks such as BinaryConnect (Courbariaux, Bengio, and David 2015), BinaryNet (Courbariaux et al. 2016), and XNOR-Net (Rastegari et al. 2016) proposed binarizing weights to $\{-1, +1\}$, using the Straight-Through Estimator (STE) (Bengio, Léonard, and Courville 2013) to enable training. BitNet (Wang et al. 2023) scaled this idea to LLMs by introducing *BitLinear* layers with binary weights, enabling addition-only inference while preserving competitive accuracy. BitNet b1.58 (Ma et al. 2024) further extended the weight set to ternary $\{-1, 0, +1\}$, improving expressiveness under the same 2-bit budget. Subsequent variants (Wang, Ma, and Wei 2025, 2024; Ma et al. 2025) and related efforts (Team et al. 2025) further advanced the practical deployment of extremely low-bit LLMs. More recently, ParetoQ (Liu et al. 2025) explores scaling laws in extremely low-bit LLM Quantization. These works inspire our approach: by extending Transformer architectures into the complex domain and adopting a symmetric, phase-aware 2-bit quantization scheme, we aim to overcome the representational inefficiencies of real-valued BitNet design while preserving their computational advantages.

2.3 Complex-Valued Neural Networks

The use of complex numbers in neural networks is not a new concept. Complex-Valued Neural Networks (CVNNs) have been explored for several decades, particularly in domains where data possesses inherent phase and magnitude properties, such as in signal processing and imaging (Hirose 2006; Lee, Hasegawa, and Gao 2022; Bassey, Qian, and Li 2021; Yang et al. 2020; Eilers and Jiang 2023). By representing weights and activations as complex numbers, CVNNs can potentially capture more intricate patterns and feature dependencies compared to their real-valued counterparts. However, the application of CVNNs to natural language processing, and specifically to LLMs, has been limited. Our work bridges this gap by demonstrating that the

complex domain offers a compelling solution to a fundamental efficiency problem in 1-bit real-valued quantization.

In contrast to existing approaches, our work integrates the advantages of complex-valued representations with efficient, extremely low-bit quantization strategies, bridging a crucial gap and opening new avenues for more efficient deployment of powerful language models.

3 The *iFairy* model

In this section, we propose *iFairy*, which extends the conventional Transformer architecture into the complex domain, and enables more expressive and efficient representations through the use of native 2-bit complex-valued weights. This section is structured as follows: Section 3.2 details the architectural adaptations required for complex-valued operations within the Transformer backbone. Section 3.3 and 3.4 then describe our quantization strategies for complex-valued weights and activations, respectively. Finally, Section 3.5 compares the computational complexity of *iFairy*, BitNet1.58, and the full-precision Llama model.

3.1 Model Architecture of *iFairy*

Our proposed model, *iFairy*, is a highly efficient Transformer designed to operate with 2-bit complex-valued weights. It leverages the rich representational capacity of the complex domain while maintaining the computational benefits of extremely low-bit quantization. The overall architecture is illustrated in Figure 1. The foundation of *iFairy* is a Complex-Valued Transformer Backbone. This backbone adapts a standard LLaMA-style architecture to the complex domain, re-engineering core components, such as the embedding layers, self-attention layers, language model head, and feed-forward networks, with ComplexLinear module, to handle complex-valued parameters and activations. This design provides the expressive power needed to learn complex data patterns. We then propose our primary PhaseQuant scheme to map the complex-valued weights of the backbone into a discrete 2-bit complex space during computation. The *iFairy* is constructed by systematically applying PhaseQuant to ComplexLinear in the complex-valued Transformer model, as shown on the right of Figure 1.

3.2 Backbone of *iFairy*

The foundational principle of our methodology is the systematic extension of the Transformer architecture to operate on complex numbers. Specifically, all model parameters and intermediate representations are complex values. A weight matrix $\mathbf{W} \in \mathbb{C}^{m \times n}$ and an input activation $\mathbf{x} \in \mathbb{C}^m$ are thus represented by their real and imaginary parts:

$$\mathbf{W} = \mathbf{W}_{\text{re}} + i\mathbf{W}_{\text{im}},$$

$$\mathbf{x} = \mathbf{x}_{\text{re}} + i\mathbf{x}_{\text{im}},$$

where $\mathbf{W}_{\text{re}}, \mathbf{W}_{\text{im}} \in \mathbb{R}^{m \times n}$, and $\mathbf{x}_{\text{re}}, \mathbf{x}_{\text{im}} \in \mathbb{R}^m$. Incorporating the mathematically robust properties of positive definiteness and conjugate symmetry, the Hermitian inner product is naturally employed for linear projection in the backbone. It is defined as:

$$\mathbf{Y} = \bar{\mathbf{x}}\mathbf{W},$$

where $\bar{\mathbf{x}}$ denotes the complex conjugate of $\mathbf{x}$. We designate the layer that performs this operation as the ComplexLinear, which serves as the complex-valued counterpart to the standard linear layer. The following subsections detail the primary modifications made to the standard Transformer architecture to realize this complex-valued backbone. A more detailed justification for these architectural choices is provided in Appendix C.

Dual-channel Projection Embedding Layers. To bridge the discrete token space and the continuous complex-valued representation space, we employ a dual-channel projecting strategy, as illustrated in the bottom part of Figure 2b. This is implemented using two parallel embedding layers. For a given input token, one layer generates the real part of the embedding vector $\mathbf{E}_{\text{re}}$, while the other generates the imaginary part $\mathbf{E}_{\text{im}}$, forming the final complex embedding $\mathbf{E} = \mathbf{E}_{\text{re}} + i\mathbf{E}_{\text{im}}$. Although structurally separate, the two pathways are implicitly coupled through the subsequent complex-valued operations, which encourage the model to learn the unified complex representation during the end-to-end training.

Efficient Complex-Valued Self-Attention. Extending the self-attention mechanism to the complex domain raises a fundamental question: What is a principled and computationally efficient way to define similarity between complex-valued queries and keys that also respects the geometric structure of complex vector spaces?

In our formulation, we adopt the real part of the Hermitian inner product as the attention score:

$$S = \text{score}(\mathbf{Q}, \mathbf{K}) = \text{Re}(\bar{\mathbf{Q}}\mathbf{K}^T) = \mathbf{Q}_{\text{re}}\mathbf{K}_{\text{re}}^T + \mathbf{Q}_{\text{im}}\mathbf{K}_{\text{im}}^T.$$

This choice ensures that all four components of $\mathbf{Q}$ and $\mathbf{K}$ —real and imaginary parts of both—are involved, thus preserving informational completeness while maintaining compatibility with standard real-valued softmax operations. Crucially, this approach admits a well-established mathematical and geometric interpretation. As discussed in (Scharnhorst 2001), the real part of the Hermitian inner product corresponds to the so-called *Euclidean angle* between complex vectors, defined by isometrically embedding the complex space $\mathbb{C}^n$ into $\mathbb{R}^{2n}$. See more in Appendix C.

The final attention output $\mathbf{O}$ is then computed by applying the softmax function to the real-valued scores and multiplying the result with the complex-valued value vectors $\mathbf{V}$:

$$\mathbf{O} = \text{softmax}\left(\frac{S}{\sqrt{d_k}}\right)\mathbf{V}.$$

For practical efficiency, we recast the complex-valued computation as a larger real-valued matrix multiplication. We concatenate the real and imaginary parts of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ along the last dimension to obtain:

$$\tilde{\mathbf{Q}} = [\mathbf{Q}_{\text{re}} \mid \mathbf{Q}_{\text{im}}], \quad \text{w.l.o.g.} \quad \tilde{\mathbf{K}}, \tilde{\mathbf{V}}$$

The score matrix can then be computed as: $S = \tilde{\mathbf{Q}}\tilde{\mathbf{K}}^T$. This transformation enables the use of highly optimized real-valued FlashAttention kernels (Dao et al. 2022; Dao 2024),

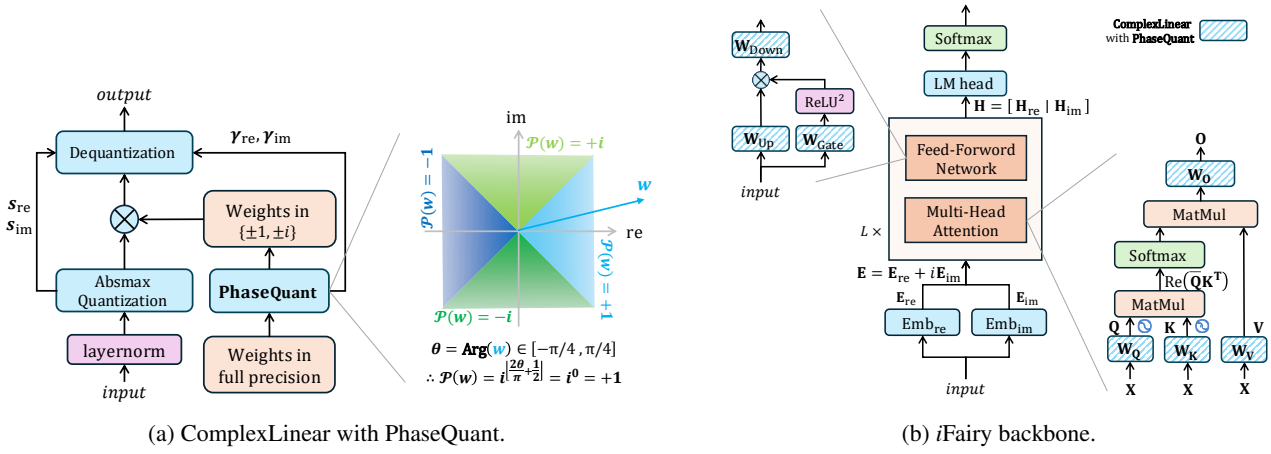


Figure 1: Overview of PhaseQuant and iFairy. The left panel illustrates the quantization process of PhaseQuant. In the right panel, PhaseQuant is applied to all major linear projections within iFairy, including $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, and $\mathbf{W}_O$ in the self-attention block, as well as $\mathbf{W}_{Up}$, $\mathbf{W}_{Gate}$, and $\mathbf{W}_{Down}$ in the feed-forward network.

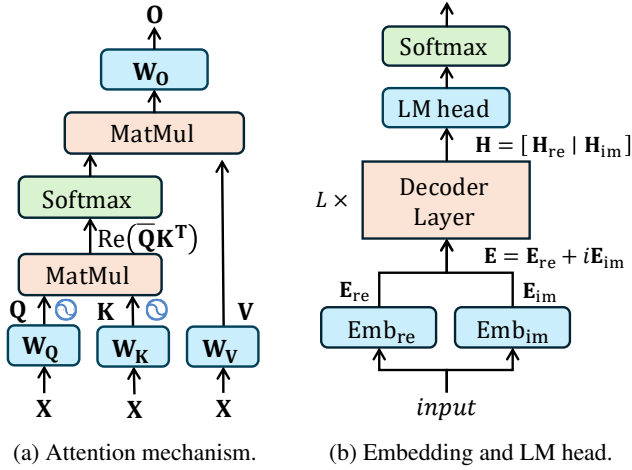


Figure 2: The complex-valued Transformer architecture.

while preserving the expressiveness of complex-valued representations. The resulting output is partitioned back into its real and imaginary components to construct the final complex-valued attention output $\mathbf{O}$. The entire procedure is summarized in Algorithm 1.

Complex-Valued Feed-Forward Network. The Feed-Forward Network (FFN) in our architecture mirrors the structure of modern LLMs like LLaMA, but operates on complex numbers. A key modification resides in the non-linear activation function. For the activation function f , we use the squared ReLU (Zhang et al. 2024), defined as $f(x) = \text{ReLU}^2(x) = (\max(0, x))^2$. Let $\mathbf{Z} = \mathbf{Z}_{re} + i\mathbf{Z}_{im}$ is the gated activation, then the application of f is:

$$f(\mathbf{Z}) = \text{ReLU}^2(\mathbf{Z}_{re}) + i \text{ReLU}^2(\mathbf{Z}_{im})$$

This design allows the network to maintain non-linearity, which is crucial for learning complex patterns, while con-

Algorithm 1: Efficient Complex-Valued Self-Attention

- 1: **Input:** Complex Query $\mathbf{Q}$, Key $\mathbf{K}$, Value $\mathbf{V}$
- 2: **Output:** Complex attention output $\mathbf{O}$
- 3: // Concatenate real and imaginary parts for $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$
- 4: **for** each matrix $\mathbf{M} \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}\}$ **do**
- 5: $\tilde{\mathbf{M}} \leftarrow [\mathbf{M}_{re} \mid \mathbf{M}_{im}]$
- 6: **end for**
- 7: // Utilize standard Flash Attention kernel
- 8: $\tilde{\mathbf{O}} \leftarrow \text{FlashAttention}(\tilde{\mathbf{Q}}, \tilde{\mathbf{K}}, \tilde{\mathbf{V}})$
- 9: $d \leftarrow$ dimension of $\tilde{\mathbf{O}}$ along the last axis
- 10: $\mathbf{O}_{re} \leftarrow \tilde{\mathbf{O}}[\dots, d/2]$
- 11: $\mathbf{O}_{im} \leftarrow \tilde{\mathbf{O}}[\dots, d/2 :]$
- 12: **return** $\mathbf{O}$

fining the operation to the real domain where such functions are well-defined and computationally inexpensive.

Complex Language Model Head. To project the final complex hidden states back to the vocabulary space for token prediction, we design our Language Model Head to be symmetric to the input embedding layer. This pattern ensures a principled and consistent mapping between the discrete token space and the continuous complex representation space at both ends of the network. As illustrated at the top of Figure 2b, the final hidden state $\mathbf{H} = \mathbf{H}_{re} + i\mathbf{H}_{im}$ is first transformed by concatenating its real and imaginary components into a single, larger real-valued matrix, $\tilde{\mathbf{H}} = [\mathbf{H}_{re} \mid \mathbf{H}_{im}]$. This matrix is then projected through a standard real-valued linear layer to compute the final logits over the vocabulary:

$$\text{logits} = \tilde{\mathbf{H}} \mathbf{W}_{out}^T,$$

where $\mathbf{W}_{out}$ is the learned weight matrix of the output projection layer. This computational form creates a strong inductive bias, encouraging the output layer to learn a projection that measures the similarity between the final hid-

den state and each vocabulary token’s latent representation using the same underlying metric that governs the model’s internal reasoning. Consequently, the symmetric output projection provides a coherent and elegant solution for the final classification step. It ensures that the transformation from complex representations to token probabilities is not an arbitrary mapping, but one that is deeply integrated with the geometric principles established throughout the entire complex-valued architecture.

Complex Rotary Position Embedding. In its original real-valued formulation, RoPE encodes absolute position by applying a rotation matrix to pairs of features in the query and key vectors. In the complex domain, this rotational logic can be implemented more directly, as a 2D rotation is equivalent to multiplication by a complex number of unit modulus, $e^{i\theta}$. Given a token at position m and a hidden dimension j , the rotary embedding is applied as follows:

$$\mathbf{q}'_{m,j} = \mathbf{q}_{m,j} e^{im\theta_j}; \mathbf{k}'_{n,j} = \mathbf{k}_{n,j} e^{in\theta_j}$$

where $\theta_j = \text{base}^{-j/d}$ is a predefined frequency, with d being the hidden dimension size. Then we have:

$$(\mathbf{q}'_m)^H \mathbf{k}'_n = \sum_{j=1}^d \bar{\mathbf{q}}_{m,j} \mathbf{k}_{n,j} e^{i(n-m)\theta_j}$$

This result shows that the attention score is modulated by a relative phase shift $e^{i(n-m)\theta_j}$ that depends solely on the position difference $n - m$. The detailed derivation is provided in Appendix C.3.

Layer Normalization. The RMSNorm is applied to real and imaginary components of activations, respectively.

3.3 PhaseQuant for Complex-Valued Weight

The core of *iFairy* lies in its quantization scheme for complex-valued weights, simulated during Quantization-Aware Training (QAT). We propose PhaseQuant, a deterministic method that maps each full-precision complex weight to one of the fourth roots of unity $\{\pm 1, \pm i\}$ based on its phase in the complex plane. Each complex weight $w = w_{\text{re}} + iw_{\text{im}}$ is first projected to a codeword using a phase-based mapping:

$$\mathcal{P}(w) = i^{\lfloor \frac{2\theta}{\pi} + \frac{1}{2} \rfloor}, \quad \theta = \text{Arg}(w) \in [-\pi, \pi].$$

Letting $w_b = \mathcal{P}(w) = w_{b,\text{re}} + iw_{b,\text{im}}$, we then compute the scaling factors γ_{re} and γ_{im} after this projection, using only the entries that are mapped to the corresponding codewords:

$$\gamma_{\text{re}} = \frac{1}{\mathbb{E}[|\mathbf{W}_{\text{re}}| \mid \mathcal{P}(\mathbf{W}) \in \{\pm 1\}]},$$

$$\gamma_{\text{im}} = \frac{1}{\mathbb{E}[|\mathbf{W}_{\text{im}}| \mid \mathcal{P}(\mathbf{W}) \in \{\pm i\}]}.$$

These factors normalize the respective components using only those entries whose phase-based projection falls into $\{\pm 1\}$ for γ_{re} or $\{\pm i\}$ for γ_{im} . Finally, the quantized value is dequantized as

$$w_q = \frac{w_{b,\text{re}}}{\gamma_{\text{re}}} + i \cdot \frac{w_{b,\text{im}}}{\gamma_{\text{im}}}.$$

During the forward pass, the full-precision weights are replaced with w_q . In the backward pass, gradients are propagated through the original weights using the Straight-Through Estimator (STE) to handle the non-differentiable quantization step. An illustration of PhaseQuant is provided in the left panel of Figure 1a.

3.4 Complex-Valued Activation Quantization

We adopt a symmetric per-token INT8 quantization scheme that processes the real ($\mathbf{x}_{\text{re}}$) and imaginary ($\mathbf{x}_{\text{im}}$) components of the activation $\mathbf{x}$ independently. For each component, a dynamic scaling factor is computed based on the maximum absolute value within the token’s feature vector. Specifically, the scaling factor for the real part is:

$$s_{\text{re}} = \frac{127}{\max(|\mathbf{x}_{\text{re}}|)}$$

The quantization function, which emulates quantization effects during training, is defined as:

$$\mathcal{Q}(\mathbf{x}, s) = \frac{1}{s} \cdot \text{round}(\text{clamp}(s \cdot \mathbf{x}, -128, 127))$$

The quantized activation is then reconstructed as: $\mathbf{x}_q = \mathcal{Q}(\mathbf{x}_{\text{re}}, s_{\text{re}}) + i \cdot \mathcal{Q}(\mathbf{x}_{\text{im}}, s_{\text{im}})$ where s_{im} is computed analogously. This per-token, component-wise quantization adapts to the varying dynamic range across tokens, yielding higher numerical precision than static per-tensor methods.

3.5 Computational Complexity Analysis

A key advantage of our approach is the enhancement of representational capacity without increasing computational overhead.

Storage Cost. The storage requirement for our model’s weights is identical to that of the 1.58-bit BitNet. Each complex weight is stored using 2 bits to represent one of the four states $\{\pm 1, \pm i\}$, thereby achieving maximum storage efficiency for a 2-bit system. The activations, quantized to INT8 for both real and imaginary parts, also follow standard low-precision data formats.

Operational Cost. While a generic complex multiplication $(a+ib)(c+id) = (ac-bd) + i(ad+bc)$ requires four real multiplications and two real additions, the computation in *iFairy* is substantially more efficient. Our quantized weights belong to a specific set that eliminates the need for multiplication. Let w_q be a quantized weight and $x_q = x_{\text{re}} + ix_{\text{im}}$ be a quantized activation. The product $\bar{x}_q \cdot w_q$ results in one of four outcomes, as summarized in Table 1.

Activation	Weight (w_q)	Result ($\bar{x}_q \cdot w_q$)
$x_q = x_{\text{re}} + ix_{\text{im}}$	+1	$x_{\text{re}} - ix_{\text{im}}$
	-1	$-x_{\text{re}} + ix_{\text{im}}$
	+i	$x_{\text{im}} + ix_{\text{re}}$
	-i	$-x_{\text{im}} - ix_{\text{re}}$

Table 1: Multiplication-free operations for the ComplexLinear layer. $\bar{x}_q$ is the conjugate of the quantized activation.

Crucially, all four operations are free of multiplications and can be implemented using additions, subtractions, and component swapping. This places the computational overhead of *iFairy* in the same class as BitNet, dominated by additions. The matrix multiplication in our Complex-Linear layer with PhaseQuant is thus transformed from a multiplication-intensive operation into an addition-intensive one.

Inference Optimization with Look-Up Tables (LUTs). The discrete nature of both our quantized weights (2-bit) and activations (INT8) makes the computation in *iFairy* highly amenable to further optimization using look-up tables (LUTs), particularly for CPU inference. Following a similar principle to optimized kernels like ‘BitNet.cpp’, the inner loop of the matrix multiplication can be significantly accelerated. For instance, a group of four 2-bit complex weights can be combined to form an 8-bit index ($4^4 = 256$ states). A LUT can be pre-computed to store the 256 possible outcomes of multiplying these four weights with a corresponding vector of four INT8 complex activations. During inference, the computation is transformed into fetching the pre-computed complex result from the LUT based on the weight configuration and accumulating it.

4 Experiments

We conduct a comprehensive empirical evaluation of *iFairy* to validate its effectiveness. We aim to answer the following key research questions:

- **RQ1 (Performance):** Does fully utilizing the 2-bit complex-valued quantization space lead to improved language modeling and downstream task performance compared to existing ternary quantization schemes?
- **RQ2 (Ablation Insights):** How do complex-valued components of the *iFairy* architecture, such as attention mechanism and LM head design, affect the performance?
- **RQ3 (Quantization Dynamics):** Does the proposed quantization scheme make effective use of the full codebook $\{\pm 1, \pm i\}$, and how do layer-wise ℓ_2 norms behave under complex-valued quantization?

4.1 Experimental Setup

We outline the setup for our empirical evaluation in this section. We first list the models and baselines, followed by the evaluation protocol, and finally the implementation details.

Models and Baselines. We evaluate our proposed model, *iFairy*, at 700M and 1.3B parameter scales. To provide a comprehensive performance context, we compare it against three primary baselines, each serving a distinct purpose:

- **Full-Precision *iFairy*:** Our own complex-valued architecture trained in full BF16 precision without quantization. This model serves as the direct upper-bound for our approach, allowing us to isolate and measure the performance impact of quantization itself.
- **FP16 LLaMA:** A standard full-precision LLaMA model, acting as a widely-accepted performance benchmark for traditional real-valued Transformers.

- **BitNet b1.58:** A 1.58-bit LLM that serves as our main low-bit competitor. We compare against both officially reported results and our reproduction based on publicly available code¹ for a comprehensive comparison. Due to computational constraints, we only reproduce the 700M variant.

Evaluation Protocol. To comprehensively assess model capabilities, our evaluation is twofold:

- **Language Modeling:** We measure perplexity (PPL) on the validation sets of WikiText2 (Merity et al. 2016) and C4 (Raffel et al. 2020). Lower PPL indicates superior language modeling ability.
- **Downstream Tasks:** We evaluate zero-shot performance on a suite of common sense reasoning tasks using the `lm-eval-harness` framework (Gao et al. 2024). The tasks include ARC-Easy (Yadav, Bethard, and Surdeanu 2019), ARC-Challenge (Yadav, Bethard, and Surdeanu 2019), Hellaswag (Zellers et al. 2019), Winogrande (Sakaguchi et al. 2021), and PIQA (Bisk et al. 2019).

Implementation Details. All models are trained from scratch under a unified setting to ensure fair comparison. We use a 100B-token corpus randomly sampled from the RedPajama-V1 dataset (Weber et al. 2024), tokenized with the LLaMA-Tokenizer². Our models are trained using the AdamW optimizer (Loshchilov and Hutter 2017) with a two-stage linear learning rate decay schedule. We divide the training process into two stages at the 50% mark. The first stage adopts standard linear learning rate scheduling with a higher peak learning rate, as 2-bit LLMs exhibit greater training stability than their full-precision counterparts. During this stage, the weight decay is set to 0.1. In the second stage, we apply a decayed learning rate schedule with a lower peak value, and the weight decay is reduced to 0. For training efficiency and stability, we employ data parallelism and a BF16 mixed-precision strategy, where we train our *iFairy* model in BF16 precision but accumulate gradients in FP32 precision. The training was conducted on a cluster of 32 NVIDIA H800 GPUs, leveraging HuggingFace Accelerate with the DeepSpeed (ZeRO Stage 1) backend. The key hyperparameters used for training *iFairy* are summarized in Table 2.

4.2 Main Results

In this section, we present the core empirical results to answer **RQ1**. We demonstrate this through a top-down analysis, starting from training dynamics, followed by language modeling perplexity, and finally, downstream task performance.

Training Dynamics and Convergence. We begin by examining the training loss, a fundamental indicator of a model’s ability to learn from data. Figure 3 compares the training loss curves of *iFairy* and BitNet b1.58. *iFairy* consistently achieves a lower training loss throughout the training process, indicating that our complex-valued quantiza-

¹<https://huggingface.co/1bitLLM>

²<https://huggingface.co/meta-llama/Llama-2-7b-hf>

Hyperparameter	Value
Learning Rate (700M)	$1.5 \times 10^{-3} \rightarrow 1.0 \times 10^{-3}$
Learning Rate (1.3B)	$1.2 \times 10^{-3} \rightarrow 0.8 \times 10^{-3}$
LR Schedule	Two-stage linear
Warmup Steps	375
Adam β_1	0.9
Adam β_2	0.95
Weight Decay	$0.1 \rightarrow 0.0$
Gradient Clipping	1.0
Batch Size	512
Sequence Length	2048
Mixed Precision	BF16

Table 2: Key training hyperparameters for *iFairy*.

tion scheme enables more effective optimization and a better fit to the training data. This superior convergence behavior lays the foundation for its strong performance on evaluation benchmarks.

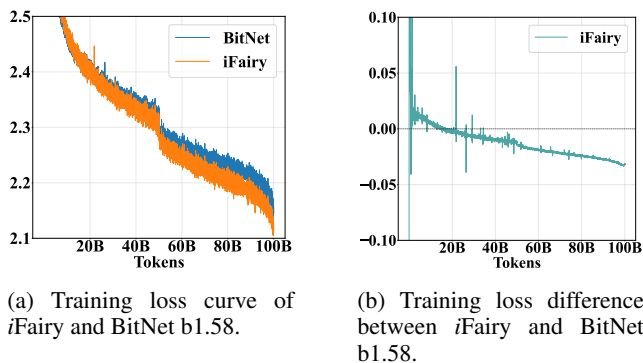


Figure 3: Training loss comparison between *iFairy* and BitNet b1.58.

Language Modeling Performance. This improved training dynamic translates directly into superior language modeling capabilities. Table 3 presents the perplexity (PPL) scores on the WikiText2 and C4 validation sets. Our method, *iFairy*, consistently outperforms both the reproduced and reported versions of BitNet b1.58 across model sizes. At the 700M scale, *iFairy* achieves an average PPL of 11.13, improving upon BitNet b1.58’s 11.51 (reproduced) and 12.87 (reported). At the 1.3B scale, *iFairy* achieves an average PPL of 10.14, significantly lower than the 11.29 reported for BitNet b1.58. These results confirm that our complex-valued, 2-bit quantization framework enhances model expressiveness under extreme compression.

Downstream Task Performance. To assess how well these improvements generalize beyond the pre-training objective, we evaluate the models on a suite of zero-shot common sense downstream tasks. The results, summarized in Table 4, highlight the strong generalization capacity of *iFairy*. Remarkably, our 1.3B *iFairy* model achieves an average accuracy of 46.52, not only exceeding the BitNet base-

Size	Model	Quant	Wiki2↓	C4↓	Avg↓
700M	FP16 LLaMA	No	–	–	12.33
	<i>iFairy</i> [◦] (Ours)	No	9.41	10.75	10.08
	BitNet b1.58*	Yes	–	–	12.87
	BitNet b1.58 [†]	Yes	10.81	12.21	11.51
	<i>iFairy</i> (Ours)	Yes	10.45	11.81	11.13
1.3B	FP16 LLaMA	No	–	–	11.25
	<i>iFairy</i> [◦] (Ours)	No	8.72	9.95	9.34
	BitNet b1.58*	Yes	–	–	11.29
	<i>iFairy</i> (Ours)	Yes	9.35	10.94	10.14

Table 3: Perplexity on WikiText2 and C4 validation sets (lower is better). * refers to the reported version in prior work (Ma et al. 2024), [†] our reproduced version, and [◦] the full-precision *iFairy*.

line but also slightly outperforming the FP16 LLaMA model (46.21). This finding underscores that the rich representations learned via our 2-bit complex quantization are highly effective and transferable to diverse downstream applications.

4.3 Ablation Studies

To dissect the sources of *iFairy*’s better performance and answer question **RQ2**, we conduct targeted ablation studies. We structure this analysis into two parts: first, we evaluate the inherent potential of a native complex-valued architecture, and second, we isolate the specific impact of our proposed quantization scheme, PhaseQuant.

Performance of Native Complex-Valued Architecture.

Before assessing our quantization scheme, it is crucial to establish the viability of a complex-valued Transformer as a strong architectural foundation. To this end, we compare our full-precision *iFairy*, the model denoted as *iFairy*[◦] in Table 3, which is trained in BF16 without any quantization, against the standard FP16 LLaMA.

As shown in Table 3 and Table 4, the native complex-valued architecture shows a distinct performance advantage. In language modeling, the full-precision *iFairy* of 700M achieves a striking average PPL of 10.08, substantially outperforming the 12.33 of the FP16 LLaMA. This superiority in core modeling capability extends to downstream generalization; *iFairy*[◦] attains a higher average accuracy of 46.18 on downstream tasks, compared to 45.51 for its real-valued counterpart. These results confirm that a native complex-valued architecture inherently possesses greater representational power than a similar-scale real-valued architecture. This not only justifies our choice of architecture but also successfully breaks through the previous informational ceiling.

Impact of Computational Pattern. A critical design choice within a complex-valued architecture is the specific computational pattern used to handle interactions between complex states. A strawman solution is to compute the attention score from the dot product of only the real parts of

Model Size	Model	Quant	ARCe $\uparrow$	ARCe $\uparrow$	HS $\uparrow$	BQ $\uparrow$	OQ $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	Avg $\uparrow$
700M	FP16 LLaMA	No	54.70	23.00	37.00	60.00	20.20	68.90	54.80	45.51
	<i>iFairy</i> [°] (Ours)	No	55.68	24.06	37.79	60.46	20.60	70.18	54.46	46.18
	BitNet b1.58 [*]	Yes	51.80	21.40	35.10	58.20	20.00	68.10	55.20	44.26
	BitNet b1.58 [†]	Yes	51.77	22.44	35.30	58.50	20.80	65.94	54.85	44.23
	<i>iFairy</i> (Ours)	Yes	53.45	23.04	36.04	57.31	21.00	68.01	54.06	44.70
1.3B	FP16 LLaMA	No	56.90	23.50	38.50	59.10	21.60	70.00	53.90	46.21
	<i>iFairy</i> [°] (Ours)	No	58.96	25.77	40.29	60.92	23.20	71.44	57.06	48.23
	BitNet b1.58 [*]	Yes	54.90	24.20	37.70	56.70	19.60	68.80	55.80	45.39
	<i>iFairy</i> (Ours)	Yes	56.65	24.66	38.69	59.60	22.20	69.80	54.06	46.52

Table 4: Zero-shot accuracy on downstream tasks. * refers to the reported version in prior work (Ma et al. 2024), [†] our reproduced version, and [°] the full-precision *iFairy*.

the query and key vectors. Similarly, the LM head projection uses only the real part of the final hidden state. As shown in the training loss comparison in Figure 4, the choice of pattern has a profound impact on training dynamics. The persistent gap between the two patterns demonstrates that our proposed computational pattern better leverages the expressive power of the complex domain by enabling the model to jointly process both real and imaginary parts throughout the network. This refined handling of information is a key contributor to the superior final performance of *iFairy*.

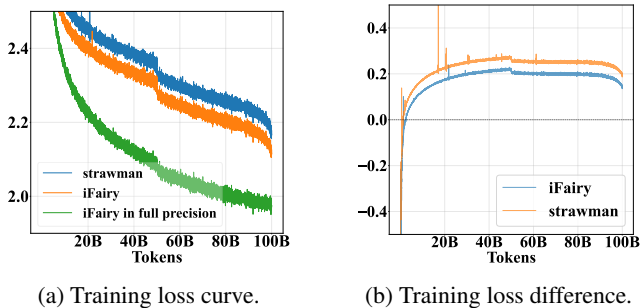


Figure 4: Training loss comparison among *iFairy*, full-precision *iFairy* and the strawman solution with simple computational pattern. We use the full-precision *iFairy* as the baseline of loss difference.

4.4 Analysis of Complex-Valued Quantized Representations

To verify that the proposed quantization scheme fully utilizes the 2-bit space, we analyze the intrinsic properties of our quantization scheme to answer our final research question **RQ3**. We examine this through three complementary perspectives: (1) the distribution of quantized weights across the 2-bit complex codebook, (2) the behavior of layer-wise weight norms, and (3) the distribution of the token embedding layer and the LM head.

Distribution of Quantized Model Weights. A key indicator of an effective multi-bit quantization scheme is its abil-

ity to leverage the entire available representational space. A poorly designed scheme might lead to representational collapse, where the model predominantly uses only a subset of the available values. We measured the empirical distribution of its quantized weights across the four complex values $\{\pm 1, \pm i\}$ to demonstrate codebook utilization of *iFairy*. As shown in Figure 5, the distribution is remarkably balanced. This near-uniformity confirms that the model effectively learns to exploit the full expressive power of the 2-bit complex codebook. Every quantum state is actively used, providing the model with the rich representational capacity that underpins its strong performance. For completeness, we include the weight distributions of other modules in Appendix D.1, which demonstrate similarly uniform usage patterns.

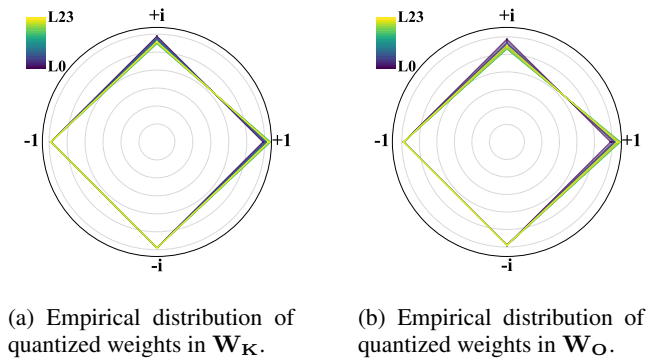


Figure 5: Quantization statistics of weight values in *iFairy*.

Layer-wise Norms of Quantized Weights. Beyond utilizing the codebook, a robust quantization method must maintain the structural integrity of the network. Unstable weight magnitudes across layers can hinder training and harm generalization. We therefore analyze the layer-wise ℓ_2 norms of the quantized weights. Figure 6 reveals that the norms remain exceptionally stable and well-distributed across all layers. This demonstrates that our method, including the separate scaling of real and imaginary components,

successfully preserves the network’s magnitude structure. Such stability is critical for maintaining healthy gradient flow throughout the deep network, preventing issues common in highly compressed models and contributing directly to the robust generalization we observed in Section 4.2. For completeness, we provide the layer-wise norm statistics for all other weight matrices, in Appendix D.2, which exhibit similar stability trends.

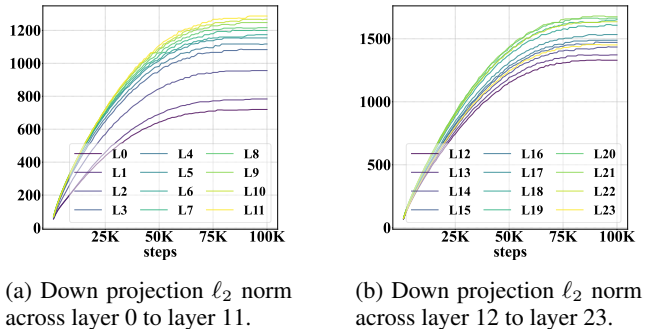


Figure 6: Layer-wise ℓ_2 norm of complex-valued quantized weights in *iFairy*. Stable norm patterns suggest good magnitude preservation and strong generalization capacity.

Distribution of Embedding Layer and LM Head. Figure 7 visualizes the distribution of token embeddings and LM head weights in the complex plane. We plot the mean-centered token embeddings in their original complex form in Figure 7a. The points exhibit an approximately uniform distribution around the origin, indicating that both real and imaginary components are utilized in a balanced manner. We apply principal component analysis (PCA) separately to the real and imaginary parts to project the embeddings and LM head weights into a 2D space in Figure 7b. The token embeddings (blue) and the LM head weights (orange) form well-separated but symmetric and coherent clusters, aligned along orthogonal directions. This structured distribution suggests that the complex-valued embedding space remains well-established and that the LM head learns to align with the token embedding. Overall, the results show that our complex-valued architecture maintains a stable and expressive embedding space.

Taken together, these analyses provide strong evidence that our complex-valued quantization scheme is both expressive and stable. The full utilization of the 2-bit codebook ensures that the model leverages the complete representational capacity available in complex space. The stable layer-wise norm dynamics indicate that the quantized weights retain well-conditioned magnitudes across the network. The structural alignment between token embeddings and the LM head demonstrates that our model has a stable and uniform embedding space in the complex domain. These properties jointly underpin the robust performance of *iFairy* under extreme bit-width constraints.

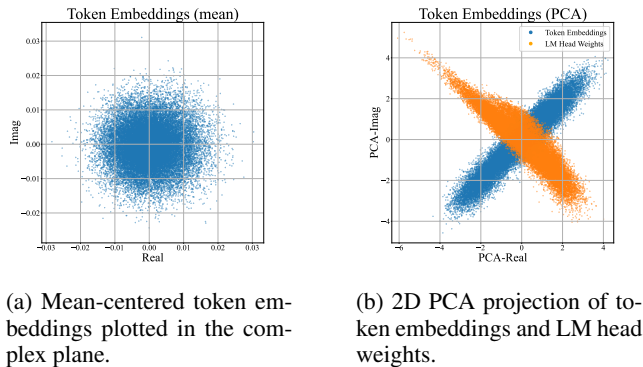


Figure 7: Visualization of Complex-Valued Token Embeddings and LM Head Weights.

5 Conclusion

We present *iFairy*, the first 2-bit complex LLM with all parameters in $\{\pm 1, \pm i\}$. We integrate complex-valued representations into the Transformer and quantize weights to the fourth roots of unity $\{\pm 1, \pm i\}$ via the proposed PhaseQuant, *iFairy* fully exploits the 2-bit space while preserving symmetry, efficiency, and hardware compatibility. Experimental results demonstrate that *iFairy* outperforms the accuracy ceiling of all existing quantization approaches under equivalent model sizes, in terms of perplexity and task accuracy.

Limitations and Future Work. Several limitations remain. First, the optimal formulation of a complex-valued attention mechanism for language modeling is still under-explored. Second, our use of separate scaling factors for the real and imaginary components may not fully preserve the original magnitude structure of complex weights. Third, deploying *iFairy* in practical systems demands careful hardware-aware design, as current CPU and GPU architectures are not optimized for complex-valued or multiplication-free computation. Future work will focus on scaling *iFairy* to larger model sizes, exploring unified or learned scaling strategies, and developing hardware accelerators tailored to complex-valued arithmetic. We also envision the design of more expressive, complex-native architectures that further enhance the benefits of complex quantization.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Bassey, J.; Qian, L.; and Li, X. 2021. A survey of complex-valued neural networks. *arXiv preprint arXiv:2101.12249*.
- Bengio, Y.; Léonard, N.; and Courville, A. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*.
- Bisk, Y.; Zellers, R.; Bras, R. L.; Gao, J.; and Choi, Y. 2019. PIQA: Reasoning about Physical Commonsense in Natural Language. In *AAAI Conference on Artificial Intelligence*.
- Bondarenko, Y.; Del Chiaro, R.; and Nagel, M. 2024. Low-rank quantization-aware training for llms. *arXiv preprint arXiv:2406.06385*.
- Chen, M.; Shao, W.; Xu, P.; Wang, J.; Gao, P.; Zhang, K.; and Luo, P. 2024. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062*.
- Courbariaux, M.; Bengio, Y.; and David, J.-P. 2015. Binaryconnect: Training deep neural networks with binary weights during propagations. *Advances in neural information processing systems*, 28.
- Courbariaux, M.; Hubara, I.; Soudry, D.; El-Yaniv, R.; and Bengio, Y. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv e-prints*, arXiv-2407.
- Eilers, F.; and Jiang, X. 2023. Building blocks for a complex-valued transformer architecture. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 1–5. IEEE.
- Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.
- Gao, L.; Tow, J.; Abbasi, B.; Biderman, S.; Black, S.; DiPofi, A.; Foster, C.; Golding, L.; Hsu, J.; Le Noac’h, A.; Li, H.; McDonell, K.; Muennighoff, N.; Ociepa, C.; Phang, J.; Reynolds, L.; Schoelkopf, H.; Skowron, A.; Sutawika, L.; Tang, E.; Thite, A.; Wang, B.; Wang, K.; and Zou, A. 2024. The Language Model Evaluation Harness.
- Hirose, A. 2006. *Complex-valued neural networks*. Springer.
- Lee, C.; Hasegawa, H.; and Gao, S. 2022. Complex-valued neural networks: A comprehensive survey. *IEEE/CAA Journal of Automatica Sinica*, 9(8): 1406–1426.
- Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, W.-M.; Wang, W.-C.; Xiao, G.; Dang, X.; Gan, C.; and Han, S. 2023. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *arXiv preprint arXiv:2306.00978*.
- Liu, Z.; Oguz, B.; Zhao, C.; Chang, E.; Stock, P.; Mehdad, Y.; Shi, Y.; Krishnamoorthi, R.; and Chandra, V. 2023. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*.
- Liu, Z.; Zhao, C.; Huang, H.; Chen, S.; Zhang, J.; Zhao, J.; Roy, S.; Jin, L.; Xiong, Y.; Shi, Y.; et al. 2025. Paretoq: Scaling laws in extremely low-bit llm quantization. *arXiv preprint arXiv:2502.02631*.
- Loshchilov, I.; and Hutter, F. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.
- Ma, S.; Wang, H.; Huang, S.; Zhang, X.; Hu, Y.; Song, T.; Xia, Y.; and Wei, F. 2025. BitNet b1. 58 2B4T Technical Report. *arXiv preprint arXiv:2504.12285*.
- Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; and Wei, F. 2024. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 1(4).
- Merity, S.; Xiong, C.; Bradbury, J.; and Socher, R. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- Miao, X.; Oliaro, G.; Zhang, Z.; Cheng, X.; Jin, H.; Chen, T.; and Jia, Z. 2023. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*.
- Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140): 1–67.
- Rastegari, M.; Ordonez, V.; Redmon, J.; and Farhadi, A. 2016. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European conference on computer vision*, 525–542. Springer.
- Sakaguchi, K.; Bras, R. L.; Bhagavatula, C.; and Choi, Y. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9): 99–106.
- Scharnhorst, K. 2001. Angles in complex vector spaces. *Acta Appl. Math.*, 69: 95–103.
- Team, M.; Xiao, C.; Li, Y.; Han, X.; Bai, Y.; Cai, J.; Chen, H.; Chen, W.; Cong, X.; Cui, G.; et al. 2025. MiniCPM4: Ultra-Efficient LLMs on End Devices. *arXiv preprint arXiv:2506.07900*.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Wan, Z.; Wang, X.; Liu, C.; Alam, S.; Zheng, Y.; Liu, J.; Qu, Z.; Yan, S.; Zhu, Y.; Zhang, Q.; et al. 2023. Efficient large language models: A survey. *arXiv preprint arXiv:2312.03863*.

Wang, H.; Ma, S.; Dong, L.; Huang, S.; Wang, H.; Ma, L.; Yang, F.; Wang, R.; Wu, Y.; and Wei, F. 2023. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*.

Wang, H.; Ma, S.; and Wei, F. 2024. BitNet a4. 8: 4-bit Activations for 1-bit LLMs. *arXiv preprint arXiv:2411.04965*.

Wang, H.; Ma, S.; and Wei, F. 2025. BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs. *arXiv preprint arXiv:2504.18415*.

Weber, M.; Fu, D.; Anthony, Q.; Oren, Y.; Adams, S.; Alexandrov, A.; Lyu, X.; Nguyen, H.; Yao, X.; Adams, V.; et al. 2024. Redpajama: an open dataset for training large language models. *Advances in neural information processing systems*, 37: 116462–116492.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, 38087–38099. PMLR.

Yadav, V.; Bethard, S.; and Surdeanu, M. 2019. Quick and (not so) dirty: Unsupervised selection of justification sentences for multi-hop question answering. *arXiv preprint arXiv:1911.07176*.

Yang, M.; Ma, M. Q.; Li, D.; Tsai, Y.-H. H.; and Salakhutdinov, R. 2020. Complex transformer: A framework for modeling complex-valued sequence. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 4232–4236. IEEE.

Zellers, R.; Holtzman, A.; Bisk, Y.; Farhadi, A.; and Choi, Y. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

Zhang, Z.; Song, Y.; Yu, G.; Han, X.; Lin, Y.; Xiao, C.; Song, C.; Liu, Z.; Mi, Z.; and Sun, M. 2024. ReLU² Wins: Discovering Efficient Activation Functions for Sparse LLMs. *arXiv preprint arXiv:2402.03804*.

A Pseudo-Code

Algorithm 2: Forward Pass during QAT of *i*Fairy

```

1: Input: Full-precision complex weight  $\mathbf{W}$ , Full-precision complex activation  $\mathbf{X}$ 
2: Output: Full-precision complex output  $\mathbf{Y}$ 
3:
4: {1. Activation Quantization-Dequantization}
5:  $s_{\text{re}} \leftarrow 127 / \max(|\mathbf{X}_{\text{re}}|)$ ;  $s_{\text{im}} \leftarrow 127 / \max(|\mathbf{X}_{\text{im}}|)$ 
6:  $\mathbf{X}_{\text{int8, re}} \leftarrow \text{round}(\text{clamp}(\mathbf{X}_{\text{re}} \cdot s_{\text{re}}, -128, 127))$ 
7:  $\mathbf{X}_{\text{int8, im}} \leftarrow \text{round}(\text{clamp}(\mathbf{X}_{\text{im}} \cdot s_{\text{im}}, -128, 127))$ 
8:  $\mathbf{X}_{\text{q, re}} \leftarrow \mathbf{X}_{\text{int8, re}} / s_{\text{re}}$ 
9:  $\mathbf{X}_{\text{q, im}} \leftarrow \mathbf{X}_{\text{int8, im}} / s_{\text{im}}$ 
10:  $\mathbf{X}_q \leftarrow \mathbf{X}_{\text{q, re}} + i \mathbf{X}_{\text{q, im}}$ 
11:
12: {2. Weight Quantization-Dequantization}
13:  $\mathbf{W}_b \leftarrow \mathcal{P}(\mathbf{W})$  {Quantized to  $\{\pm 1, \pm i\}$ }
14:  $\gamma_{\text{re}} \leftarrow 1 / \mathbb{E}[|\mathbf{W}_{\text{re}}| \mid \mathcal{P}(\mathbf{W}) \in \{\pm 1\}]$ 
15:  $\gamma_{\text{im}} \leftarrow 1 / \mathbb{E}[|\mathbf{W}_{\text{im}}| \mid \mathcal{P}(\mathbf{W}) \in \{\pm i\}]$ 
16:  $\mathbf{W}_{\text{q, re}} \leftarrow \mathbf{W}_{\text{b, re}} / \gamma_{\text{re}}$ 
17:  $\mathbf{W}_{\text{q, im}} \leftarrow \mathbf{W}_{\text{b, im}} / \gamma_{\text{im}}$ 
18:  $\mathbf{W}_q \leftarrow \mathbf{W}_{\text{q, re}} + i \mathbf{W}_{\text{q, im}}$ 
19:
20: {3. Perform Complex Linear Operation}
21:  $\mathbf{Y}' \leftarrow \overline{\mathbf{X}_q} \cdot \mathbf{W}_q$  {Note the conjugate on  $\mathbf{X}_q$ }
22:
23: {4. Straight-Through Estimator (STE)}
24: {In backward pass, gradient flows to original  $\mathbf{W}$ }
25:  $\mathbf{Y} \leftarrow \mathbf{Y}'$ 
26: return  $\mathbf{Y}$ 

```

B Training Details

B.1 Hardware and Software Configuration

All models were trained on a high-performance computing cluster equipped with $32 \times$ NVIDIA H100 Tensor Core GPUs (80 GB HBM3 memory per GPU). Training was performed with DeepSpeed ZeRO Stage 1 for optimizer state sharding and memory efficiency, and bf16 mixed precision was used to reduce memory footprint and improve computational throughput while maintaining numerical stability.

B.2 Hyperparameters

The detailed model configurations for different parameter scales are listed in Table 5. Both model sizes share the same sequence length (2048 tokens) and are trained on 100B tokens, but differ in hidden dimension, gated linear unit (GLU) expansion size, attention head count, and total parameter count.

C Theoretical Justification

C.1 Justification for Self-Attention Mechanism.

In our complex-valued attention mechanism, we adopt the real part of the Hermitian inner product as the attention score:

$$S = \text{Re}(\overline{\mathbf{Q}}\mathbf{K}^\top).$$

This choice is both mathematically principled and practically motivated.

From a geometric perspective, this formulation corresponds to the so-called *Euclidean angle* between complex vectors, as discussed by Scharnhorst (Scharnhorst 2001). Given a complex vector space $V_{\mathbb{C}} \cong \mathbb{C}^n$, one can isometrically embed it into a real vector space $V_{\mathbb{R}} \cong \mathbb{R}^{2n}$ by splitting each complex coordinate into its real and imaginary parts. Under this embedding, the real part of the Hermitian inner product becomes:

$$\text{Re}(\langle \mathbf{a}, \mathbf{b} \rangle_{\mathbb{C}}) = \mathbf{a}_{\text{re}}^\top \mathbf{b}_{\text{re}} + \mathbf{a}_{\text{im}}^\top \mathbf{b}_{\text{im}} = (\mathbf{A}, \mathbf{B})_{\mathbb{R}},$$

which is exactly the standard dot product in $\mathbb{R}^{2n}$. Therefore, the real part retains the familiar interpretation of *directional alignment* via projection, analogous to the cosine similarity in real-valued spaces.

More generally, the real part of the Hermitian product appears as the *real component* of the so-called *complex angle* between vectors, defined by:

$$\cos \Theta_c(\mathbf{a}, \mathbf{b}) = \frac{(\mathbf{a}, \mathbf{b})_{\mathbb{C}}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in \mathbb{C},$$

which can be written as $\cos \Theta_c = \rho e^{i\varphi}$, where $\rho = |\cos \Theta_c|$ is the *Hermitian angle* and φ is the *pseudo-angle*. Scharnhorst demonstrates that the Euclidean angle serves as a natural projection of this complex-valued structure onto the real line. Specifically, he shows that:

$$\cos \Theta_c(\mathbf{a}, \mathbf{b}) = \cos \Theta(\mathbf{a}, \mathbf{b}) + i \cos \Theta_K(\mathbf{a}, \mathbf{b}) \sin \Theta(\mathbf{a}, \mathbf{b}),$$

where Θ is the Euclidean angle and Θ_K is the so-called *Kähler angle*. Taking only the real part of the complex angle thus corresponds precisely to using $\cos \Theta(\mathbf{a}, \mathbf{b})$.

This projection not only simplifies implementation by avoiding the need to handle complex-valued scores in softmax, but also preserves the essential geometric information—i.e., how much the vectors are aligned in phase. Since any similarity score in the attention mechanism must ultimately be a real-valued scalar to interface with softmax, $\text{Re}(\overline{\mathbf{Q}}\mathbf{K}^\top)$ emerges as the most natural, interpretable, and structure-preserving choice. It also avoids the ambiguity associated with the phase term φ , which lacks geometric meaning in projective settings.

In conclusion, using only the real part of the Hermitian inner product offers a geometrically faithful similarity measure, consistent with classical constructions of angles in complex vector spaces (Scharnhorst 2001), while remaining computationally compatible with attention kernels tailored for modern GPUs.

C.2 Activation Function in FFN

The position-wise Feed-Forward Network (FFN) in our architecture adapts the structural principles of modern LLMs like LLaMA to operate entirely within the complex domain. Critically, the choice of the non-linear activation function presents a trade-off between sparsity and gradient smoothness. For instance, the standard Rectified Linear Unit (ReLU) enforces strong sparsity but suffers from a non-differentiable point at the origin. Conversely, functions like

Table 5: Model configurations for *iFairy*

Size	Hidden Size	GLU Size	#Heads	#Layers	Batch Size	#Tokens	Seq Length
700M	1536	4096	16	24	1M tokens	100B	2048
1.3B	2048	5460	32	24	1M tokens	100B	2048

SwiGLU offer smoother gradients at the cost of sacrificing this beneficial hard sparsity. To resolve this trade-off, we employ the Squared ReLU (ReLU²) activation function (Zhang et al. 2024), defined for a real input x as:

$$f(x) = \text{ReLU}^2(x) = (\max(0, x))^2.$$

For a complex pre-activation $\mathbf{Z} = \mathbf{Z}_{\text{re}} + i\mathbf{Z}_{\text{im}}$, the function is applied component-wise:

$$f(\mathbf{Z}) = \text{ReLU}^2(\mathbf{Z}_{\text{re}}) + i \text{ReLU}^2(\mathbf{Z}_{\text{im}})$$

This function retains the sparsity of ReLU, as its output is zero for the identical set of non-positive inputs. Additionally, the function’s derivative, $\frac{d}{dx} \text{ReLU}^2(x) = 2 \cdot \text{ReLU}(x)$, is continuous across its entire domain, including the origin, which resolves the abrupt gradient change in ReLU and contributes to a more stable optimization landscape. In parallel, the quadratic nature of ReLU² has a valuable inductive bias by enhancing activation contrast. It suppresses weak positive signals while amplifying strong ones, promoting a more decisive and robust feature selection mechanism. Therefore, ReLU² uniquely synthesizes the benefits of high sparsity with smoother gradients and enhanced feature representation, establishing it as an ideal choice for our complex-valued backbone.

C.3 Derivation of Complex RoPE Embedding

In its original real-valued formulation, RoPE encodes absolute position by applying a rotation matrix to pairs of features in the query and key vectors. Specifically, it pairs adjacent dimensions to simulate 2D rotations. However, in the complex domain, this logic can be implemented more directly and uniformly: a 2D rotation is equivalent to multiplication by a complex exponential of unit modulus, $e^{i\theta}$.

Given a token at position m and hidden dimension index j , we define the complex rotary embedding as:

$$\mathbf{q}'_{m,j} = \mathbf{q}_{m,j} \cdot e^{im\theta_j}, \quad \mathbf{k}'_{n,j} = \mathbf{k}_{n,j} \cdot e^{in\theta_j},$$

where $\theta_j = \text{base}^{-j/d}$ is a predetermined frequency, and d is the hidden dimension size.

Now consider the Hermitian inner product between the transformed query $\mathbf{q}'_m$ and key $\mathbf{k}'_n$:

$$\begin{aligned} (\mathbf{q}'_m)^H \mathbf{k}'_n &= (\mathbf{q}_m \odot e^{im\Theta})^H (\mathbf{k}_n \odot e^{in\Theta}) \\ &= \sum_{j=1}^d \overline{(\mathbf{q}_{m,j} e^{im\theta_j})} (\mathbf{k}_{n,j} e^{in\theta_j}) \\ &= \sum_{j=1}^d \overline{\mathbf{q}_{m,j}} \overline{e^{im\theta_j}} (\mathbf{k}_{n,j} e^{in\theta_j}) \\ &= \sum_{j=1}^d \overline{\mathbf{q}_{m,j}} \mathbf{k}_{n,j} e^{-im\theta_j} e^{in\theta_j} \\ &= \sum_{j=1}^d \overline{\mathbf{q}_{m,j}} \mathbf{k}_{n,j} e^{i(n-m)\theta_j} \end{aligned}$$

where $\odot$ denotes element-wise multiplication and Θ is the vector of frequencies $[\theta_1, \dots, \theta_d]$. The key properties used are the conjugate of a product ($\overline{ab} = \overline{a}\overline{b}$) and the conjugate of a complex exponential ($\overline{e^{i\phi}} = e^{-i\phi}$).

This result shows that the attention score is modulated by a relative phase shift $e^{i(n-m)\theta_j}$ that depends solely on the position difference $n - m$. Therefore, relative positional information is directly encoded into the inner product in a rotation-equivariant manner. Unlike the real-valued RoPE approach, which requires pairing dimensions to simulate complex rotation, the complex-valued version applies rotation directly and uniformly to each feature dimension. This leads to a more natural, expressive, and mathematically clean positional encoding mechanism for complex-valued models.

C.4 Design of PhaseQuant

Our phase-based quantization scheme is motivated by principles from information theory and complex geometry, offering both representational efficiency and geometric stability.

Full Information Capacity. From an information-theoretic perspective, our goal is to maximize the information encoded within the allocated 2-bit budget. While the ternary set $\{-1, 0, +1\}$ used in BitNet yields approximately 1.58 bits of entropy, our quaternary set $\{\pm 1, \pm i\}$ achieves the theoretical maximum of $\log_2(4) = 2$ bits under a uniform distribution. This ensures full utilization of each bit, enhancing the expressiveness of the quantized model.

Geometric Robustness and Symmetry. Our quantization points are chosen as the 4th roots of unity on the complex unit circle. These points are equidistant and symmetrically distributed in the complex plane, maximizing the angular

separation between centroids. Such symmetry not only provides geometric robustness and maximal separation in Euclidean distance, but also facilitates uniform treatment of directions, leading to more stable optimization dynamics and improved error resilience.

Preservation of Directional Information. In complex-valued neural networks, the phase primarily encodes directional information, while the magnitude reflects importance. Since we employ per-tensor scaling factors $(\gamma_{re}, \gamma_{im})$ to approximate magnitude, our quantization can focus on accurately preserving phase. This aligns with the intuition that directional information is more critical in many learning tasks, especially in low-bit regimes.

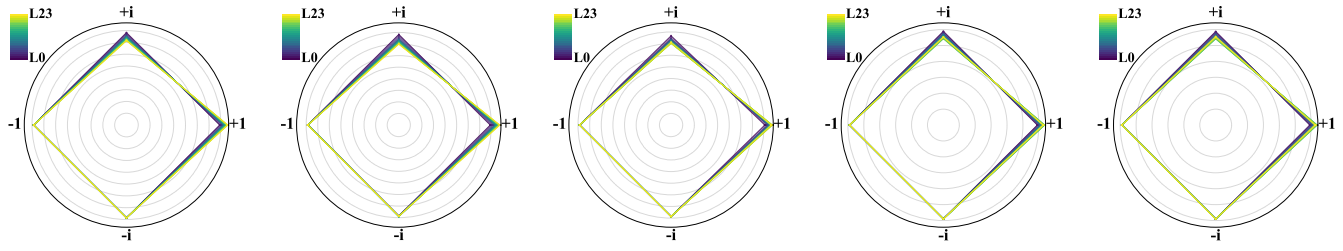
D Additional Experimental Results

D.1 Quantized Weight Distributions

To complement the main analysis in Section 4.4, we visualize the empirical distributions of quantized weights for all major parameter matrices which is not shown in the main text, including $\mathbf{W}_Q$ and $\mathbf{W}_V$ in the self-attention block, as well as $\mathbf{W}_{Up}$, $\mathbf{W}_{Gate}$, and $\mathbf{W}_{Down}$ in the feed-forward network. Figure 8 shows the usage frequency of the four complex values $\{\pm 1, \pm i\}$ across these modules. All components exhibit balanced or near-uniform distributions, confirming that *iFairy* consistently avoids representational collapse and fully exploits the 2-bit codebook throughout the model.

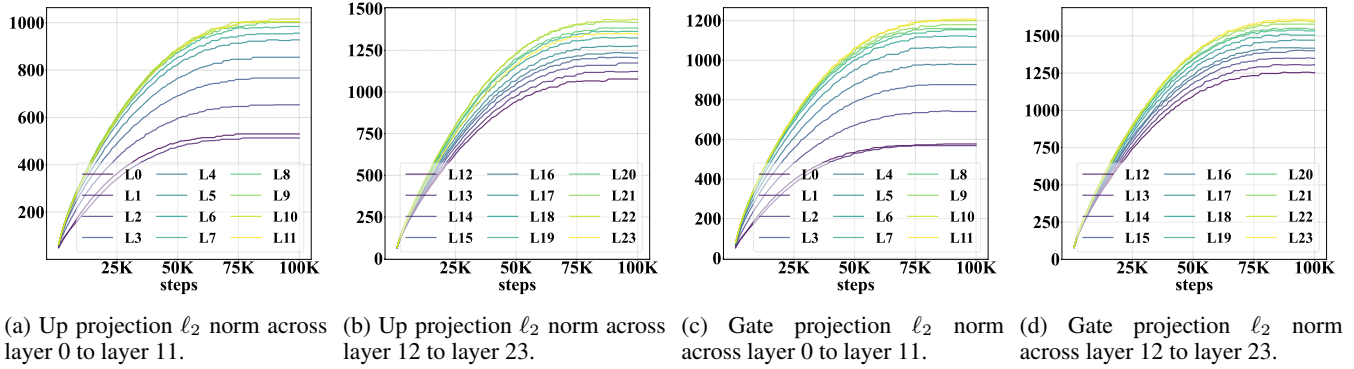
D.2 Layer-wise Norms

To complement the analysis in Section 4.4, we provide the ℓ_2 norms of all quantized weight matrices across layers. As shown in Figure 9, these components exhibit similarly stable norm distributions, further confirming that our quantization scheme consistently preserves the scale structure across the entire model.

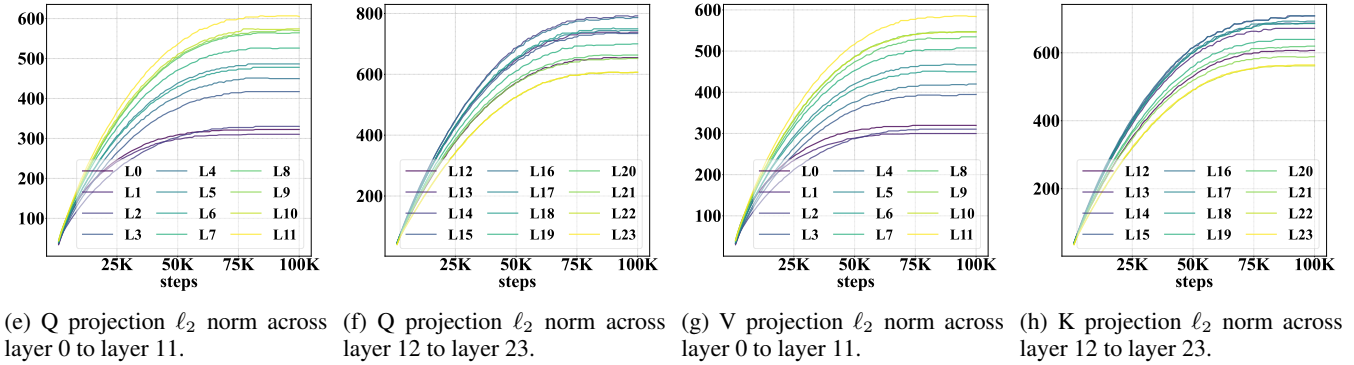


(a) Empirical distribution of quantized weights in $\mathbf{W}_{Up}$. (b) Empirical distribution of quantized weights in $\mathbf{W}_{Gate}$. (c) Empirical distribution of quantized weights in $\mathbf{W}_{Down}$. (d) Empirical distribution of quantized weights in $\mathbf{W}_Q$. (e) Empirical distribution of quantized weights in $\mathbf{W}_V$.

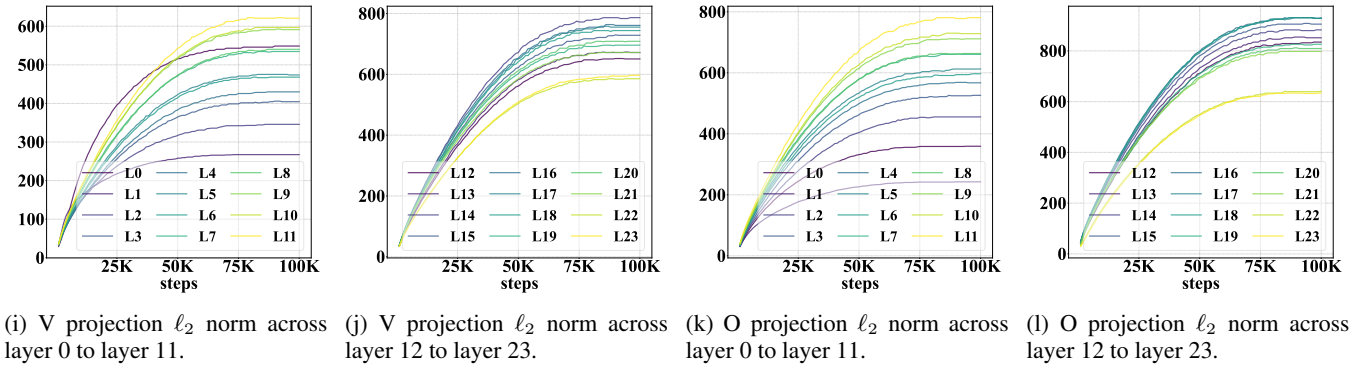
Figure 8: Quantization statistics of weight values in *iFairy*.



(a) Up projection ℓ_2 norm across layer 0 to layer 11. (b) Up projection ℓ_2 norm across layer 12 to layer 23. (c) Gate projection ℓ_2 norm across layer 0 to layer 11. (d) Gate projection ℓ_2 norm across layer 12 to layer 23.



(e) Q projection ℓ_2 norm across layer 0 to layer 11. (f) Q projection ℓ_2 norm across layer 12 to layer 23. (g) V projection ℓ_2 norm across layer 0 to layer 11. (h) K projection ℓ_2 norm across layer 12 to layer 23.



(i) V projection ℓ_2 norm across layer 0 to layer 11. (j) V projection ℓ_2 norm across layer 12 to layer 23. (k) O projection ℓ_2 norm across layer 0 to layer 11. (l) O projection ℓ_2 norm across layer 12 to layer 23.

Figure 9: Layer-wise ℓ_2 norm of complex-valued quantized weights in *iFairy*.